

07

Regularni izrazi

Biologija je puna šablona: proteinski domeni, motivi, mesta sečenja, degenerisani prajmeri, poli-A repovi

Šabloni u biološkim podacima

U sekvencama

- proteinski domeni
- vezivni motivi transkripcionih faktora
- mesta sečenja restrikcioni enzima
- mesta degenerisanih PCR prajmera
- nizovi istog nukleotida

U drugim podacima

- lokacije mapiranja očitavanja
- geografske koordinate uzoraka
- taksonomska imena i imena gena
- pristupni brojevi gena
- rezultati BLAST pretrage

Fiksni niz nije dovoljan

- Kolika je dužina poli-A repa?
- Deo pristupnog broja između 3. znaka i donje crte
- Da li protein ima domen sa mnogo varijacija?

Modul re, sirovi stringovi i re.search

Moduli

Specijalizovani alati se učitavaju naredbom `import`, a njihove funkcije pozivamo sa prefiksom imena modula.

```
import re
re.search(pattern, string)
```

Sirovi stringovi

Slovo `r` ispred navodnika isključuje specijalne karaktere poput `\n` i `\t`, kojih u šablonima ima mnogo.

```
print(r"\t\n")
# ispisuje: \t\n
```

```
dna = "ATCGCGAATTCAC"
if re.search(r"GAATTC", dna):
    print("restriction site found!")
```

IZLAZ

```
restriction site found!
```

`re.search(šablon, string)` proverava da li se šablon javlja bilo gde u stringu.

Alternacija i grupe karaktera

ŠABLON	ZNAČENJE	PRIMER
<code>GG(A T)CC</code>	A ili T na tom mestu	AvaII: GGACC i GGTCC
<code>GC[ATGC]GC</code>	jedan od navedenih karaktera	BisI: GCNGC, N je bilo koja baza
<code>GC.GC</code>	tačka: bilo koji karakter	poklapa i GCFG C, GC9GC
<code>[^ATGC]</code>	bilo koji karakter osim navedenih	nejednoznačne baze: N, R, Y...

```
dna = "ATCGCGAATTCAC"  
if re.search(r"GG(A|T)CC", dna):  
    print("restriction site found!")
```

Jedan šablon umesto složenog uslova `re.search(r"GGACC", dna)` or `re.search(r"GGTCC", dna)`.

Kvantifikatori i pozicije

SIMBOL	ZNAČENJE	ŠABLON	POKLAPA
?	0 ili 1 put	GAT?C	GATC, GAC
+	1 ili više puta	GGGA+TTT	GGGATTT, GGGAAATTT; ne GGGTTT
*	0 ili više puta	GGGA*TTT	GGGTTT, GGGATTT, GGGAAATTT
{n}	tačno n puta	GA{5}T	GAAAAAT
{n,m}	od n do m puta	GA{2,4}T	GAAT, GAAAT, GAAAAT
^	početak stringa	^AAA	AAATTT; ne GGGAAATTT
\$	kraj stringa	GGG\$	AAAGGG; ne AAAGGGCCC

Kvantifikator važi za prethodni karakter ili grupu u zagradama: **GGG(AAA)?TTT** poklapa GGGAAATTT i GGGTTT.

Kombinovanje: kompletna eukariotska iRNK

^

početak stringa

ATG

start kodon

[ATGC]{30,1000}

30 do 1000 baza A, T, G ili C

A{5,10}

poli-A rep: 5 do 10 A

\$

kraj stringa

Više zapisa, isti šablon

GG(A|T)CC · GG[AT]CC · (GGACC|GGTCC) ·
(GGA|GGT)CC · G{2}[AT]C{2}

Birajte zapis koji je najlakše pročitati.

re.search ili re.match?

re.search traži poklapanje bilo gde u stringu, a re.match samo od njegovog početka. Najčešće nam treba re.search.

Šta se poklopilo i gde: match objekat



Poklapanje počinje na indeksu 2 i završava se pre indeksa 13. Svetlo: fiksni delovi šablona (GA, AC, AC). Narandžasto: delovi uhvaćeni zagradama, `group(1)` = CGT i `group(2)` = GT.

```
dna = "ATGACGTACGTACGACTG"
m = re.search(r"GA([ATGC]{3})AC([ATGC]{2})AC", dna)
print("entire match: " + m.group())
print("first bit: " + m.group(1))
print("second bit: " + m.group(2))
print("start: " + str(m.start()))
print("end: " + str(m.end()))
```

IZLAZ

```
entire match: GACGTACGTAC
first bit: CGT
second bit: GT
start: 2
end: 13
```

Pozicije celog poklapanja i pojedinih grupa

```
dna = "ATGACGTACGTACGACTG"  
m = re.search(r"GA([ATGC]{3})AC([ATGC]{2})AC", dna)  
print("start: " + str(m.start()))  
print("end: " + str(m.end()))  
print("group one start: " + str(m.start(1)))  
print("group one end: " + str(m.end(1)))  
print("group two start: " + str(m.start(2)))  
print("group two end: " + str(m.end(2)))
```

IZLAZ

```
start: 2  
end: 13  
group one start: 4  
group one end: 7  
group two start: 9  
group two end: 11
```

Broj grupe kao argument metoda `start` i `end` daje pozicije te grupe. Ovde bismo ih mogli i izračunati, ali kod šablona promenljive dužine to nije moguće.

Deljenje i višestruka poklapanja

```
# re.split: deli string na mesta poklapanja  
dna = "ACTNGCATRGCTACGTYACGATSCGAWTCG"  
print(re.split(r"[^ATGC]", dna))
```

LISTA DELOVA

```
['ACT', 'GCAT', 'GCTACGT', 'ACGAT', 'CGA', 'TCG']
```

```
# re.findall: lista svih poklapanja  
dna = "ACTGCATTATATCGTACGAAATTATACGCGCG"  
print(re.findall(r"[AT]{4,100}", dna))
```

STRINGOVI, BEZ POZICIJA

```
['ATTATAT', 'AAATTATA']
```

```
# re.finditer: match objekti, pa i pozicije  
for match in re.finditer(r"[AT]{3,100}", dna):  
    print(match.start(), match.end())
```

POČETAK I KRAJ

```
5 12  
18 26
```

Postavka i prvi kriterijumi

```
accs = ["xkn59438", "yhdck2", "eihd39d9", "chdsye847",
        "hedle3455", "xjhd53e", "45da", "de37dp"]
for acc in accs:
    if re.search(r"5", acc):
        print(acc)
```

Lista i petlja su lak deo. Za svaki kriterijum menjamo samo šablon u `re.search`.

KRITERIJUM	ŠABLON	ISPISUJE
sadrži cifru 5	5	xkn59438 hedle3455 xjhd53e 45da
sadrži d ili e	(d e)	yhdck2 eihd39d9 chdsye847 hedle3455 xjhd53e 45da de37dp
d, pa negde posle e	d.*e	chdsye847 hedle3455 xjhd53e de37dp

Pažljivo sa kvantifikatorima

KRITERIJUM	ŠABLON	ISPISUJE
zamka: d, pa e	d.+e	chdsye847 hedle3455 xjhd53e (fali de37dp)
d, tačno jedan znak, pa e	d.e	hedle3455
d i e, bilo kojim redom	d.*e, e.*d	eihd39d9 chdsye847 hedle3455 xjhd53e de37dp

```
for acc in accs:
    if re.search(r"d.*e", acc) or re.search(r"e.*d", acc):
        print(acc)
```

+ traži bar jedan znak između, pa promašuje de37dp, gde e sledi odmah posle d. Zato je potrebno *.

Početak, kraj i ponavljanja

KRITERIJUM	ŠABLON	ISPISUJE
počinje sa x ili y	<code>^(x y)</code>	xkn59438 yhdck2 xjhd53e
počinje sa x ili y i završava se sa e	<code>^(x y).*e\$</code>	xjhd53e
tri ili više cifara zaredom	<code>\d{3,}</code>	xkn59438 chdsye847 hed1e3455
završava se sa d, pa a, r ili p	<code>d[arp]\$</code>	45da de37dp

`\d` je skraćenica za grupu svih cifara `[0123456789]`, a `{3,}` znači „tri ili više”, bez gornje granice.

Korak 1: od mesta prepoznavanja do šablona

AbcI

ANT*AAT → A[ATGC]TAAT

N je bilo koja baza; seče 3 baze posle početka.

AbcII

GCRW*TG → GC[AG][AT]TG

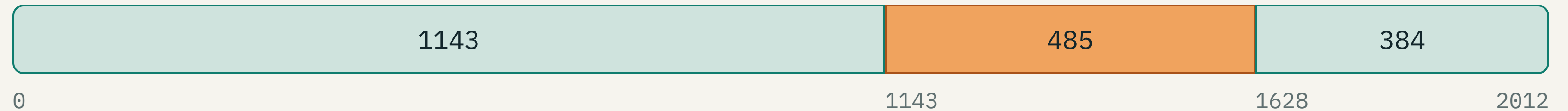
R je A ili G, W je A ili T; seče 4 baze posle početka.

```
import re
dna = open("dna.txt").read().rstrip("\n")
print("AbcI cuts at:")
for match in re.finditer(r"A[ATGC]TAAT", dna):
    print(match.start() + 3)
```

AbcI cuts at:
1143
1628

`start()` daje početak motiva (1140, 1625); enzim seče 3 baze dalje.

Korak 2: lista svih mesta sečenja



```
all_cuts = [0]
for match in re.finditer(r"A[ATGC]TAAT", dna):
    all_cuts.append(match.start() + 3)
all_cuts.append(len(dna))
print(all_cuts)
```

[0, 1143, 1628, 2012]

„Zamišljena” mesta
sečenja na početku (0) i na
kraju (dužina).

Korak 3: od mesta sečenja do dužina

```
for i in range(1, len(all_cuts)):  
    this_cut_position = all_cuts[i]  
    previous_cut_position = all_cuts[i-1]  
    fragment_size = this_cut_position - previous_cut_position  
    print("one fragment size is " + str(fragment_size))
```

```
one fragment size is 1143  
one fragment size is 485  
one fragment size is 384
```

`range(1, len(all_cuts))`: počinjemo od drugog elementa, jer prvi nema prethodnika.

Radimo sa indeksima, a ne sa elementima: element `i` i njegov prethodnik `i-1`.

Dužina fragmenta je razlika dva susedna mesta sečenja.

Korak 4: dva enzima i sortiranje

```
all_cuts = [0]
for match in re.finditer(r"A[ATGC]TAAT", dna):
    all_cuts.append(match.start() + 3)
for match in re.finditer(r"GC[AG][AT]TG", dna):
    all_cuts.append(match.start() + 4)
all_cuts.append(len(dna))
sorted_cuts = sorted(all_cuts)
```

BEZ SORTIRANJA

```
[0, 1143, 1628, 488, 1577, 2012]
1143, 485, -1140, 1089, 435
```

SA SORTED

```
[0, 488, 1143, 1577, 1628, 2012]
488, 655, 434, 51, 384
```

Računanje dužina pretpostavlja rastući redosled mesta sečenja. Negativan fragment je jasan znak da nešto nije u redu; **sorted** to rešava.

Rezime i vežbe

Šta smo naučili

- Moduli i import; sirovi stringovi
- re.search, a re.match samo od početka
- Alternacija, grupe karaktera, kvantifikatori, pozicije
- group, start i end match objekta
- re.split, re.findall i re.finditer

Vežbe

1. Od pristupnih brojeva ispišite one koji ispunjavaju svaki kriterijum posebno:

xkn59438, yhdck2, eihd39d9, chdsye847, hedle3455, xjhd53e, 45da, de37dp

sadrže 5 · sadrže d ili e · sadrže d pa e · d i e sa jednim slovom između · sadrže i d i e · počinju sa x ili y · počinju sa x ili y i završavaju se sa e · tri ili više cifara zaredom · završavaju se sa d pa a, r ili p

2. Dvostruka digestija: predvidite dužine fragmenata sekvence iz dna.txt posle sečenja izmišljenim enzimima AbcI (ANT*AAT) i AbcII (GCRW*TG); * je mesto sečenja.